PROYECTO UNIVERSITARIO DE DESARROLLO DE SOFTWARE • INFORME TÉCNICO

Sistema de Gestión Operativa, Médica y Financiera — Bienestar App

Arquitectura de Software, Stack Tecnológico, Desglose Estructural de Carpetas y Marco de Contexto Operativo para Inteligencia Artificial.

Entidad Patrocinante:	Comité de Bienestar — Ilustre Municipalidad de Peralillo
Marco Legal Chileno:	Ley N° 19.754 (Constitución de Servicios y Comités de Bienestar Municipal)
Tipo de Documento:	Informe Técnico de Arquitectura, Tecnologías y Manual de Contexto IA
Arquitectura:	Clean Architecture desacoplada (Frontend Next.js + Backend FastAPI + PostgreSQL)
Estado de Implementación:	Operativo y Validado (Módulos 1 al 5 desplegados en Docker)
Fecha de Generación:	11 de November de 2026 — Versión 1.0 Oficial

Propósito de este Informe

RESUMEN EJECUTIVO: Este documento recopila la totalidad del conocimiento arquitectónico, tecnológico y funcional del sistema *Bienestar App*. Explica cómo la plataforma digitaliza la postulación a cuotas de salud, la rendición de boletas médicas con caducidad a 30 días, el cálculo de bonos indexados a la escala municipal (Grado 15) y la auditoría inmutable de decisiones directivas. Asimismo, consagra las directrices y reglas inviolables para que modelos de Inteligencia Artificial (IA) intervengan el código con precisión quirúrgica y absoluto respeto por la coherencia del negocio.

1. Origen, Contexto y Marco Legal de la Aplicación

El sistema **Bienestar App** surge de la necesidad real y concreta de digitalizar los procesos asistenciales, previsionales y contables del Comité de Bienestar de la **Ilustre Municipalidad de Peralillo**, región del Libertador General Bernardo O'Higgins, Chile.

1.1. Fundamentación Legal: Ley N° 19.754

En Chile, la **Ley N° 19.754** autoriza a las municipalidades y a su personal a constituir comités o servicios de bienestar con el fin de proporcionar prestaciones que contribuyan al bienestar del funcionario y de sus cargas familiares reconocidas. La ley fija elementos estructurales que rigen esta aplicación:

- **Financiamiento Dual:** Se compone por el aporte del funcionario mediante un descuento por planilla mensual (fijado en 4% del sueldo base) y el aporte institucional municipal (subvención anual calculada sobre el padrón activo en UTM).
- **Autonomía Directiva:** El Comité es administrado por un Directorio paritario/colegiado encabezado por el Alcalde o su representante (Presidente) y representantes electos de los funcionarios (Secretario, Tesorero, Directores y Auditor).
- **Período de Carencia:** Exigencia estatutaria de al menos 6 meses de permanencia continua cotizando para tener derecho a la cobertura de prestaciones y beneficios económicos.

1.2. Problemáticas del Proceso Tradicional y Solución Tecnológica

Previo a la conceptualización de este proyecto, la administración de bienestar sufría de fricción operativa crítica:

1. **Falta de Visibilidad del Socio:** Los funcionarios desconocían su cupo de salud disponible ($150.000 / $250.000) y debían consultar manualmente a la directiva.
2. **Riesgo de Prescripción:** El reglamento impone que los gastos médicos prescriben a los 30 días corridos; la recepción física en papel provocaba rechazos por demoras de tramitación.
3. **Sobregiros Financieros:** Al no existir reserva preventiva en tiempo real, múltiples solicitudes en revisión podían superar el fondo disponible asignado.
4. **Carencia de Auditoría Legal:** La aprobación de cheques y subsidios carecía de traza digital inmutable apta para revisiones de Contraloría General de la República.

Valor Público y Operativo

Impacto del Sistema: Bienestar App centraliza en un único portal la autosuficiencia del socio ('Hoja Socio'), el control estricto de fechas y comprobantes con visor integrado, la retención preventiva automática y una bitácora inmutable (AuditLog) que registra cada clic resolutivo del Directorio.

2. Stack Tecnológico y Justificación de Herramientas

La selección del stack tecnológico se rigió por estándares de la industria que garantizan tipado estricto, alto rendimiento asíncrono, mantenibilidad a largo plazo y facilidad de despliegue mediante contenedores:

Capa / Subsistema	Tecnología Seleccionada	Versión	Justificación de Arquitectura y Beneficio
Frontend Framework	`Next.js (App Router)`	14.1+	Enrutamiento basado en carpetas, renderizado optimizado, Server/Client components desacoplados y rápida respuesta UI.
Lenguaje Frontend	`TypeScript`	5.4+	Tipado estático robusto, prevención de errores en tiempo de compilación y consistencia con los contratos del backend.
Diseño & UI	`Tailwind CSS`	3.4+	Diseño modular responsivo con paleta cromática oficial de la I. Municipalidad de Peralillo, mobile-first.
Backend API	`FastAPI (Python)`	0.110+	Framework asíncrono de alto rendimiento (ASGI/Starlette), documentación interactiva OpenAPI/Swagger automática.

Validación de Datos	**`Pydantic`**	v2.6+	Validación estricta de esquemas, sanitización de entradas, control de fechas pasadas/futuras y parseo ultrarrápido.
ORM & Persistencia	**`SQLAlchemy + Asyncpg`**	2.0+	Mapeo objeto-relacional asíncrono moderno, sesiones seguras no bloqueantes y consultas SQL optimizadas.
Motor de BD	**`PostgreSQL`**	15 Alpine	Base de datos relacional estándar del sector público con soporte completo de transacciones ACID y tipos Enums.
Orquestación	**`Docker & Compose`**	Compose v2	Despliegue unificado y reproducible en cualquier entorno Windows/Linux con volumen persistente para BD y uploads.
Seguridad & Auth	**`JWT + Passlib (Bcrypt)`**	HS256	Autenticación sin estado (stateless), cifrado unidireccional de contraseñas y control de acceso basado en roles (RBAC).

3. Arquitectura del Software y Flujo de Datos

El proyecto se diseñó bajo los principios de **Clean Architecture** (Arquitectura Limpia) y separación de responsabilidades, estructurado como un *Monolito Modular Desacoplado*. Esta estructura previene el acoplamiento indebido y permite que el núcleo de las reglas de negocio no dependa de los detalles del framework.

3.1. Modelo de Capas Arquitectónicas

Capa del Sistema	Ubicación Física	Responsabilidad Primaria
1. Presentación (UI)	frontend/src/app frontend/src/components	Captura la interacción del usuario, renderiza vistas responsivas, ejecuta validaciones visuales inmediatas y envía peticiones HTTP tipadas.
2. Controladores API	backend/app/api/v1	Expone endpoints RESTful, valida permisos y roles mediante inyección de dependencias (require_roles) y serializa respuestas.
3. Servicios de Negocio	backend/app/services	Contiene las reglas funcionales puras: cálculo de carencia, validación de caducidad de 30 días, retención preventiva y cálculo de bonos sobre Grado 15.
4. Validación de Dominio	backend/app/schemas	Modelos Pydantic v2 que formalizan las entradas y salidas, impidiendo inyecciones o datos fuera de rango normativo.
5. Persistencia & ORM	backend/app/models backend/app/db	Entidades SQLAlchemy mapeadas a tablas relacionales con integridad referencial, índices y transacciones asíncronas seguras.

3.2. Ciclo de Vida de una Transacción Operativa (Ejemplo Reembolso HU-03 / HU-04)

A modo de ejemplo representativo del flujo desacoplado, cuando un socio solicita un reembolso médico:

1. **Verificación Previa:** El frontend consulta el resumen previsional. Si el socio tiene menos de 6 meses de antigüedad, la interfaz bloquea el formulario e informa la carencia.
2. **Validación Interactiva:** Mientras el usuario digita la fecha del documento, Next.js llama a POST /reembolsos/validar-fecha para advertir si superó los 30 días corridos.
3. **Retención Preventiva:** Al presionar 'Enviar', el backend verifica el cupo y resta inmediatamente el monto del saldo_salud_disponible, registrando la solicitud en estado en_revision.
4. **Carga de Respaldo:** En un segundo paso continuo, se envía el comprobante tributario (PDF/Imagen) y se almacena en el volumen seguro /app/uploads/reembolsos.
5. **Resolución y Auditoría:** La Directiva examina el archivo en el visor embebido. Si aprueba, se consolida el gasto y emite cheque. Si rechaza con motivo, **el servicio restituye automáticamente el cupo retenido** al socio y registra la traza en audit_logs.

4. Anatomía y Estructura Integral de Carpetas

Para que cualquier desarrollador o modelo de Inteligencia Artificial entienda de inmediato dónde reside cada responsabilidad, a continuación se detalla la función precisa de cada directorio del proyecto:

Directorio / Archivo	Entorno	Función y Contenido Técnico Detallado
`backend/main.py`	Backend	Punto de entrada de FastAPI. Configura CORS, eventos de ciclo de vida (lifespan para inicializar BD) y registra las rutas base /api/v1.
`backend/app/api/`	Backend	**Controladores REST:** deps.py inyecta dependencias de base de datos y control de roles; v1/endpoints/ contiene los routers organizados por módulo (afiliacion, reembolsos, beneficios, finanzas, admin, auth).
`backend/app/core/`	Backend	**Configuración Central:** config.py gestiona variables de entorno con Pydantic Settings; security.py maneja tokens JWT y hash Bcrypt.

`backend/app/db/`	Backend	**Gestión de Base de Datos:** session.py provee el motor y sesión asíncrona; init_db.py ejecuta migraciones idempotentes y crea los usuarios semilla.
`backend/app/models/`	Backend	**Entidades ORM:** Declaración de tablas SQLAlchemy (User, SolicitudAfiliacion, SolicitudReembolso, BonoCatalogo, SolicitudBeneficio, AuditLog).
`backend/app/schemas/`	Backend	**Contratos Pydantic:** Validación de esquemas Request/Response, tipos estrictos y validadores de reglas de negocio para entrada/salida.
`backend/app/services/`	Backend	**Lógica Pura de Negocio:** Motores de cálculo independientes de HTTP (reembolsos, topes, cálculo de bonos sobre Grado 15, envíos SMTP).
`frontend/src/app/`	Frontend	**Rutas App Router:** Vistas principales de la aplicación web: / (Dashboard), /afiliacion, /reembolsos, /aprobaciones, /beneficios, /finanzas, /admin, /login.
`frontend/src/components/`	Frontend	**Componentes UI Modulares:** Organizados por dominio (admin, afiliacion, beneficios, reembolsos, ui) para máxima reutilización y limpieza visual.
`frontend/src/context/`	Frontend	**Estado Global:** AuthContext.tsx encapsula la sesión del usuario, token JWT, carga de perfil y métodos de inicio y cierre de sesión.
`frontend/src/layouts/`	Frontend	**Estructura Visual:** Componentes de envoltura estructural como Sidebar.tsx, Header.tsx y MainLayout.tsx.
`frontend/src/lib/`	Frontend	**Utilidades Cliente:** api.ts provee clientes fetch tipados para endpoints y carga multipart; utils.ts formatea CLP y valida RUT chileno.
`frontend/src/types/`	Frontend	**Contratos TypeScript:** index.ts contiene todas las interfaces de dominio sincronizadas exactamente con los esquemas del backend.
`docs/`	Transversal	**Documentación Viva:** Especificaciones de historias de usuario (HU-01 a HU-10), cuestionarios con el cliente y supuestos reglamentarios.
`CONTEXTO_IA.md`	Raíz	**Manual de Contexto IA:** Guía de referencia operativa inmediata para que cualquier modelo de lenguaje sepa cómo intervenir el código.

5. Módulos Funcionales y Reglas de Negocio (HU-01 a HU-10)

El sistema implementa 10 Historias de Usuario (HU) agrupadas en 5 módulos funcionales con estricto apego a la normativa municipal chilena:

5.1. Módulo 1: Afiliación, Socios y Cargas Familiares (HU-01 y HU-02)

- **Postulación en Línea (HU-01):** Formulario digital sin sesión previa para funcionarios municipales (Planta y Contrata). Período reglamentario de recepción de solicitudes: enero a junio.
- **Descuento por Planilla:** Es requisito indispensable autorizar expresamente el descuento del 4% del sueldo base mensual.
- **Acreditación de Cargas:** Registro de cargas legales con verificación de RUN y carga de certificados civiles de respaldo (Nacimiento, Matrimonio, Acuerdo de Unión Civil).
- **Resolución de Directorio (HU-02):** Solo Presidente o Secretario pueden aprobar/rechazar solicitudes en sesión formal con N° de Acta. Al aprobar, el sistema calcula la fecha de término de carencia (+6 meses) y crea automáticamente la cuenta de usuario.

5.2. Módulo 2: Reembolsos Médicos y Salud (HU-03 y HU-04)

- **Bloqueo por Carencia:** Exige un mínimo estricto de 6 meses de permanencia continua cotizando. El sistema calcula y proyecta la fecha en que quedará habilitado.
- **Caducidad de 30 Días Corridos:** Ningún gasto médico, dental o farmacéutico puede superar los 30 días corridos de antigüedad. El sistema valida esta regla en tiempo real.
- **Tope Anual y Retención Preventiva:** El socio dispone de un fondo anual ($150.000). Al solicitar un reembolso, el monto se retiene de forma inmediata para evitar sobregiros.
- **Bandeja de Aprobaciones y Restitución (HU-04):** La Directiva examina el comprobante en un visor embebido de alta resolución. Si se rechaza, el motivo es obligatorio y el sistema restituye de inmediato el saldo retenido.

5.3. Módulo 3: Beneficios Sociales y Bonos (HU-05 y HU-06)

- **Indexación Dinámica al Grado 15:** Los bonos se calculan como un porcentaje directo del Sueldo Imponible de Referencia del Grado 15 Municipal (configurable, por defecto $750.000 CLP).
- **Catálogo de Beneficios (HU-05):** Bono Vacaciones (15% G15), Bono Escolaridad (8% G15 con cert. de alumno regular hasta los 24 años), Bono Nacimiento (20% G15), Fallecimiento de Afiliado (50% G15), Catástrofes / Incendios (50% G15) y Enfermedad Catastrófica (100% G15).
- **Liquidación Masiva de Calendario (HU-06):** Módulo para ejecutar el Bono de Invierno (15% en agosto) y el Bono de Navidad (10% a 15% según acuerdo de Directorio en diciembre) mediante nóminas masivas.

5.4. Módulo 4: Control Financiero y Aportes Municipales (HU-07 y HU-08)

- **Nómina de Retenciones RRHH (HU-07):** Generación del listado consolidado de socios activos con el 4% autorizado para cruce con Remuneraciones y cuenta corriente municipal.
- **Fórmula de Aporte Municipal (HU-08):** Cálculo de la subvención institucional anual aplicando la fórmula reglamentaria: *Socios Activos al 31 de Diciembre* × *4 UTM* × *Valor UTM a la fecha del decreto*.

5.5. Módulo 5: Seguridad, Perfiles y Auditoría Inmutable (HU-09 y HU-10)

- **Estructura de Roles de Directiva (HU-09):** Cuentas con privilegios específicos: Presidente, Secretario, Tesorero, Directiva/Admin, Auditor/Revisor y Socio Activo. Máximo 7 cuentas directivas con voto.
- **Bitácora Inmutable (AuditLog):** Registro append-only que graba RUN, nombre, rol, IP, timestamp, módulo y detalle ante cualquier resolución sensible. No puede ser borrada ni modificada.
- **Hoja de Socio (HU-10):** Portal de transparencia donde cada afiliado monitorea en vivo su cartola histórica de salud, transacciones, montos retenidos y estado previsional.

6. Protocolo y Manual de Actuación para Inteligencia Artificial

Este capítulo define las directrices que debe seguir cualquier agente de Inteligencia Artificial que intervenga el repositorio en el futuro a fin de preservar la arquitectura y las reglas del negocio:

Directriz Normativa para la IA	Regla de Implementación Inviolable
1. Consulta Previa Obligatoria	Antes de modificar código, la IA debe leer CONTEXTO_IA.md para alinear cualquier cambio con los supuestos normativos ya confirmados con el cliente.
2. Integridad de Auditoría	Toda nueva acción resolutiva o cambio de estado sensible debe invocar a registrar_auditoria() insertando una fila inmutable en audit_logs.
3. Sincronización de Tipos	Si se modifica un esquema Pydantic en backend/app/schemas, se debe actualizar obligatoriamente su interfaz TypeScript equivalente en frontend/src/types/index.ts.
4. Convención Monetaria Chilena	La moneda es el Peso Chileno (CLP), sin decimales, con punto de miles (ej: $150.000). Usar siempre la utilidad cliente formatCurrency() de @/lib/utils.
5. Validación de RUN / RUT	Los identificadores de personas deben normalizarse con guion y dígito verificador. Usar las funciones formatRun y isValidRun de @/lib/utils.
6. Estilo Didáctico	Mantener los comentarios con la marca [ESTUDIANTE INFORMÁTICA] explicando con claridad conceptual qué hace cada bloque y qué permite en el sistema.
7. Registro de Decisiones	Al completar cualquier cambio relevante, la IA debe añadir una fila al historial de cambios al pie de CONTEXTO_IA.md documentando la intervención.

7. Credenciales de Prueba, Conclusiones y Próximos Pasos

7.1. Cuentas Semilla Preconfiguradas para Pruebas (Ambiente de Desarrollo)

El archivo backend/app/db/init_db.py inicializa de forma automática e idempotente las siguientes cuentas de prueba para validar los distintos perfiles de acceso:

Rol Funcional	Nombre y Cargo	RUT de Acceso	Correo Institucional	Clave Inicial
Superusuario	Super Administrador (TI)	`11.111.111-1`	superadmin@mperalillo.cl	`Super#2026`
Presidente	María Elena Soto	`12.345.678-9`	msoto@mperalillo.cl	`Presi#2026`
Secretario	Patricia Fuentes R.	`13.456.789-0`	pfuentes@mperalillo.cl	`Secre#2026`
Tesorero	Roberto Morales P.	`14.982.112-4`	rmorales@mperalillo.cl	`Teso#2026`
Admin / Directiva	Jorge Contreras L.	`16.234.567-8`	jcontreras@mperalillo.cl	`Admin#2026`
Revisor / Auditor	Andrea Ibarra V.	`17.345.678-K`	aibarra@mperalillo.cl	`Audit#2026`
Socio Activo (>6m)	Carlos Valenzuela M.	`15.489.123-K`	cvalenzuela@mperalillo.cl	`Socio#2026`
Socia Nueva (<6m)	Francisca Sepúlveda T.	`19.876.543-2`	fsepulveda@mperalillo.cl	`Socio#2026`

7.2. Conclusiones Técnicas del Proyecto Universitario

La arquitectura implementada para el **Comité de Bienestar de la I. Municipalidad de Peralillo** demuestra la viabilidad y potencia de integrar un stack moderno (Next.js 14 + FastAPI + PostgreSQL) bajo principios de alta cohesión y bajo acoplamiento. La separación estricta entre presentación, servicios de negocio y persistencia relacional garantiza que el sistema no solo resuelva las exigencias académicas de la carrera, sino que constituya una solución de calidad corporativa lista para operar en un entorno municipal real.

Síntesis Final

ESTADO DEL ARTE Y CONTINUIDAD: Gracias a la creación simultánea del archivo CONTEXTO_IA.md y este informe técnico, el proyecto queda totalmente preparado para recibir futuras iteraciones, integraciones con el servidor de correos institucional (SMTP) o pasarelas de pago bancario de BancoEstado, manteniendo la misma rigurosidad técnica y legal.